

**INSTITUTO FEDERAL DO PIAUÍ – CAMPUS PAULISTANA
TECNÓLOGO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
MATÉRIA: INTELIGÊNCIA ARTIFICIAL**

FELIPE PEREIRA COELHO

**CLASSIFICAÇÃO NO DATASET CIFAR-100
UMA ABORDAGEM FOCADA NO CONTROLE DE OVERFITTING EM CENÁRIOS DE
DADOS ESCASSOS**

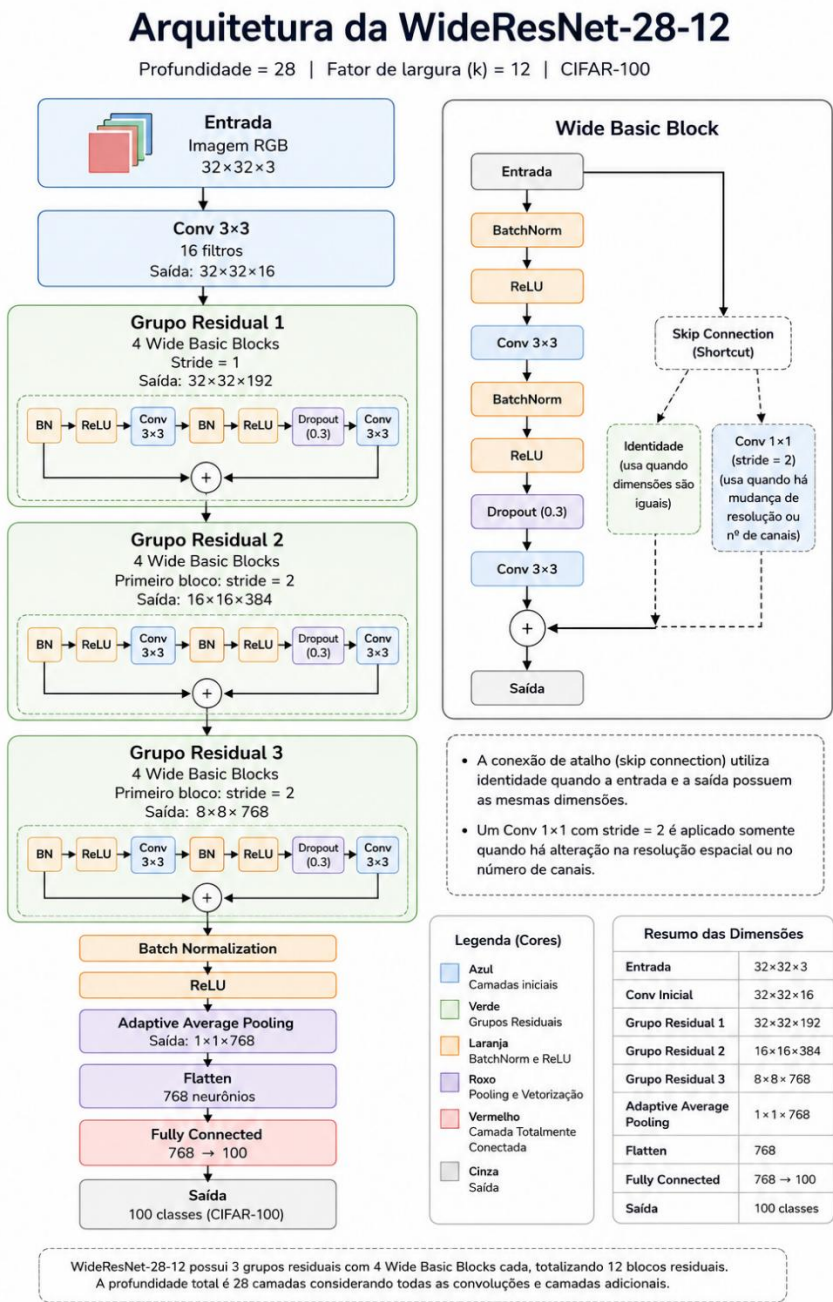
PAULISTANA -PI

2026

1. Introdução

Este trabalho desenvolve uma Rede Neural Convolucional para classificar o CIFAR-100: 100 classes finas, com apenas 500 imagens de treino por classe. Esse é o dado mais importante para entender todas as decisões deste relatório: **pouco dado por classe** significa que o risco central não é "o modelo não aprender", mas sim "o modelo decorar o treino em vez de generalizar" (overfitting). Praticamente toda escolha de arquitetura e hiperparâmetro discutida abaixo foi feita em função desse risco — e os resultados finais (85,08% Top-1) servem como evidência de que o equilíbrio encontrado funcionou.

2. Arquitetura da CNN:



Escolhi uma **Wide ResNet-28-12**, implementada do zero em PyTorch, em vez de uma CNN sequencial simples (convolução → pooling → convolução → pooling → densa). Duas decisões de design principais explicam essa escolha:

Por que conexões residuais, e não uma rede sequencial:

Para separar bem 100 classes finas eu precisava de uma rede com capacidade razoável — o que normalmente significa empilhar muitas camadas. O problema é que em redes muito profundas o gradiente se degrada ao passar por muitas camadas (vanishing gradient): as camadas iniciais recebem um sinal de erro cada vez mais fraco e o treino trava. As conexões residuais (out + shortcut, em cada WideBasicBlock) resolvem isso dando ao gradiente um "atalho" para pular blocos inteiros. Isso é o que **permitiu** usar uma rede grande o suficiente (28 camadas com peso) sem sofrer o problema de treinabilidade que uma rede sequencial da mesma profundidade teria.

Por que "larga" (mais filtros) em vez de "mais funda" (mais camadas):

Dado que eu já tinha o problema do pouco dado por classe, aumentar a profundidade indefinidamente pioraria o overfitting sem necessariamente ajudar a generalização — mais camadas = mais parâmetros = mais capacidade de decorar o treino. A alternativa "larga" (fator de largura 12, ou seja, 12x mais filtros por camada que uma ResNet convencional) dá capacidade de representação suficiente com uma profundidade moderada (28), o que na prática treina mais estável e mais rápido do que uma rede fina e muito profunda com capacidade equivalente.

Downsampling via stride, sem MaxPooling:

A redução espacial ($32 \times 32 \rightarrow 16 \times 16 \rightarrow 8 \times 8$) é feita por convolução com stride 2, e não por uma camada de pooling separada. Isso não foi uma escolha arbitrária: convolução com stride aprende *como* reduzir a resolução (os pesos da convolução decidem quais informações preservar), enquanto MaxPooling aplica uma regra fixa (pega o valor máximo). Em troca disso, o modelo perde a garantia de invariância a pequenas translações que o MaxPooling dá "de graça" — por isso o Random Crop e o Random Erasing no pipeline de augmentation (seção 4) ajudam a compensar essa ausência, forçando robustez a deslocamentos por outra via.

3. Hiperparâmetros: o porquê de cada valor

Batch size = 512 → por que, e o que isso força a mudar.

Escolhi um batch grande principalmente pela estabilidade das estatísticas do Batch Normalization: o BN calcula média e variância das ativações *dentro do batch*, e com batch pequeno essas estimativas ficam ruidosas, o que desestabiliza o treino de uma rede tão funda. Batch grande também aproveita melhor o paralelismo da GPU. O custo é que, com apenas ~97 atualizações de peso por época (50.000/512), cada atualização precisa "valer mais" — e é exatamente isso que me levou à próxima decisão.

Learning rate de pico = 0,2 → consequência direta do batch size.

Como o batch é 4x maior que o valor de referência mais comum na literatura para CIFAR (batch 128, $lr \approx 0,05-0,1$), escalei a lr proporcionalmente (regra de escalonamento linear de Goyal et al., 2017) para compensar o menor número de atualizações por época. Sem esse ajuste, o modelo teria dado passos pequenos demais para o pouco número de atualizações disponíveis, e provavelmente não teria convergido dentro de 150 épocas.

Warmup de 5 épocas → por que não aplicar 0,2 desde o início.

Uma lr alta (0,2) aplicada direto sobre pesos ainda aleatórios (logo após a inicialização Kaiming) tende a produzir passos grandes em direções não confiáveis, arriscando divergência — ainda mais combinando batch grande + BN, cujas estatísticas também começam "cruas". O warmup sobe a lr linearmente nas 5 primeiras épocas, dando tempo para os pesos e as estatísticas do BN saírem do estado aleatório antes de se expor à lr máxima.

Cosine annealing → por que a lr precisa descer, e como isso se vê no gráfico.

Nas épocas iniciais (após o warmup), passos grandes ajudam o modelo a explorar rapidamente o espaço de pesos. Mas perto de um mínimo da função de perda, passos grandes fazem o otimizador "pular por cima" do mínimo em vez de assentar nele — a loss fica oscilando sem convergir de fato. O cosine decay reduz a lr suavemente ao longo do treino para resolver exatamente isso. **Esse efeito é visível no próprio gráfico de loss de treino:** entre as épocas ~20 e ~130 a loss oscila bastante (entre 2,5 e 3,2) — período em que a lr ainda está relativamente alta e o MixUp/CutMix (ver seção 4) mantêm os alvos "ruidosos". É só perto do fim do schedule, quando a lr já caiu bastante, que o modelo consegue de fato assentar.

Dropout 0,3 e Weight Decay 5e-4 → resposta direta ao "pouco dado por classe".

Essas duas técnicas atacam o mesmo problema (overfitting) por caminhos diferentes: o Dropout desliga 30% das ativações aleatoriamente a cada passo, impedindo a rede de depender demais de neurônios específicos; o weight decay penaliza pesos grandes na função de perda, mantendo os pesos "moderados" e evitando que o modelo ajuste-se de forma excessivamente fina aos exemplos de treino. Com apenas 500 imagens por classe, sem essas duas restrições a rede de ~52M de parâmetros teria capacidade de sobra para memorizar o treino quase perfeitamente e generalizar mal.

Label smoothing 0,1 → por que não usar rótulos "duros".

Rótulos 0/1 "puros" incentivam o modelo a ficar excessivamente confiante nas próprias predições (empurrando os logits para valores extremos), o que tende a piorar a generalização e a calibração das probabilidades de saída. Suavizar os alvos (10% de "dúvida" distribuída entre as classes erradas) desestimula esse excesso de confiança — coerente com a mesma lógica de controle de overfitting das demais técnicas.

Early stopping (paciência 30) → por que não simplesmente rodar as 150 épocas fixas.

Definir 150 como teto e deixar o early stopping decidir a parada real evita duas situações ruins: parar cedo demais (perdendo ganhos que ainda viriam) ou continuar treinando além do ponto em que a validação para de melhorar (o que tende a piorar a generalização mesmo com a loss de treino ainda caindo).

4. Técnicas de Data Augmentation e Regularização: por que essa combinação

Random Crop + Flip + AutoAugment + Random Erasing: cada uma ataca uma forma diferente de o modelo "colar" no treino: dependência de posição exata (Crop), de orientação (Flip), padrões de cor/contraste específicos das imagens de treino (AutoAugment), e dependência de uma única região da imagem para reconhecer a classe (Random Erasing). Como discutido na seção 2, o Crop e o Erasing também compensam parcialmente a ausência de MaxPooling na arquitetura.

MixUp e CutMix, desligados na época 135 → por que essa técnica e por que desligar no fim.

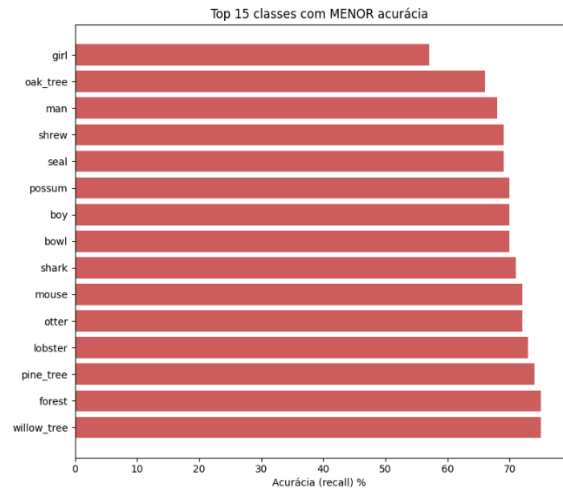
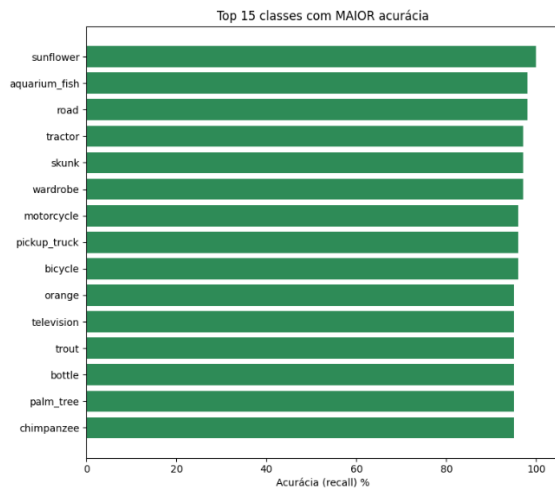
Misturar pares de imagens/rótulos suaviza as fronteiras de decisão entre classes, mais uma camada de defesa contra overfitting num dataset pequeno. Mas essa técnica tem um custo: como os rótulos viram alvos "soft" (misturas), o modelo nunca é exposto aos rótulos reais e "puros" durante essa fase — por isso a loss de treino fica artificialmente alta e instável enquanto o MixUp/CutMix está ativo (visível no platô de loss entre as épocas ~20–130 do gráfico). Desligar essa técnica nas últimas ~15 épocas permite que o modelo finalmente otimize diretamente para os rótulos reais, o que **explica diretamente** a queda abrupta de loss de ~2,2 para ~1,1 observada por volta da época 135 — e é razoável supor que essa fase final de refinamento contribuiu diretamente para o salto de desempenho refletido nos 85,08% de acurácia final.

5. Resultados

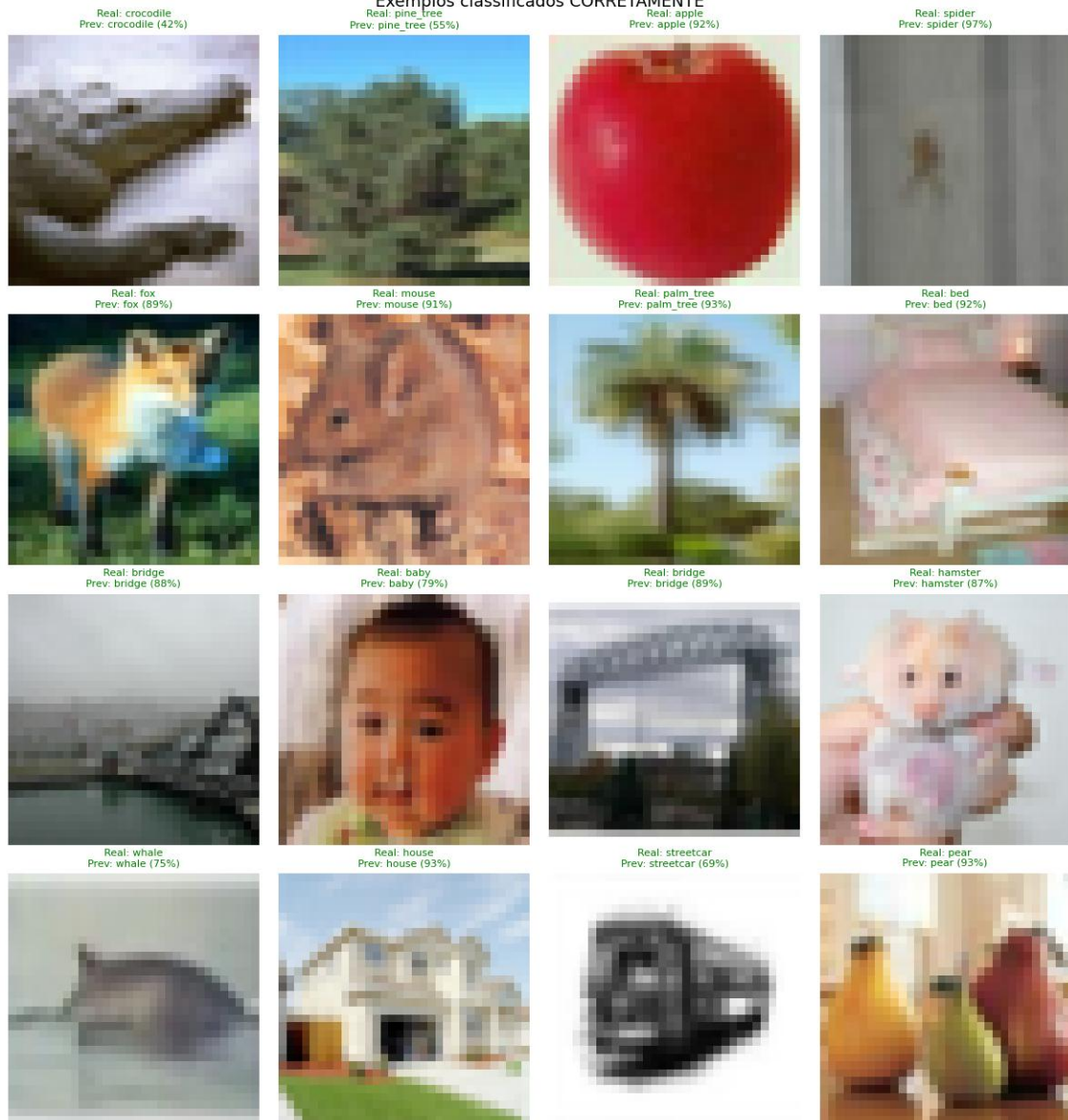
Métrica	Valor
Loss	63.85%
Top-1 Accuracy	85.08%
Top-5 Accuracy	97.03%
Macro Precision	85.25%
Macro Recall	85.08%
Macro F1-Score	85.08%

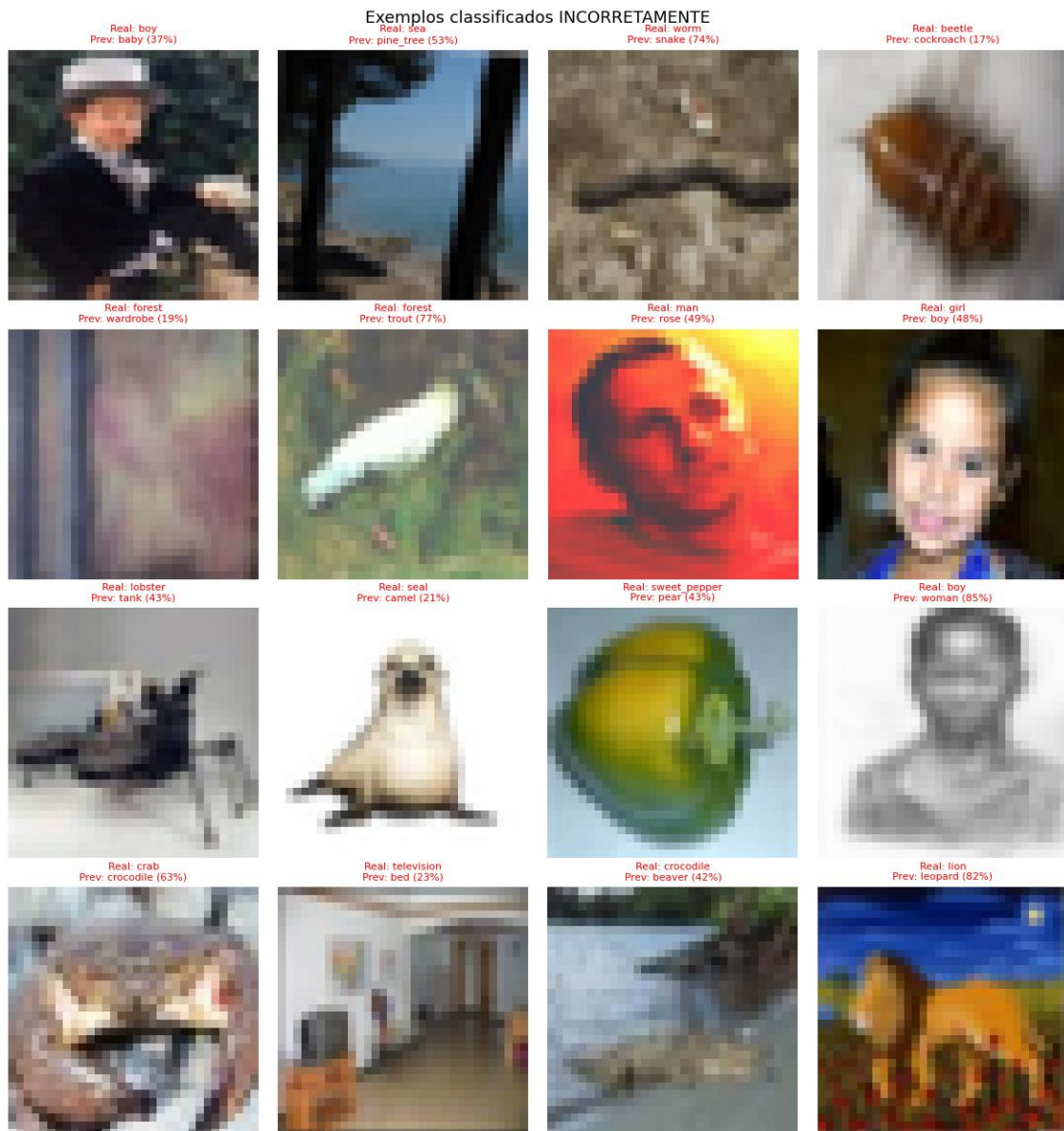
Ligando os resultados às escolhas feitas: o equilíbrio entre Precision (85,25%) e Recall (85,08%) macro — praticamente idênticos — é um indício de que o conjunto de técnicas de regularização (Dropout, weight decay, label smoothing) funcionou como pretendido: o modelo não está enviesado para prever poucas classes com muita confiança às custas de outras, o que seria esperado se o overfitting não tivesse sido bem controlado dado o pouco volume de dados por classe.

A Top-5 Accuracy de 97,03%, bem acima da Top-1, sugere que a maior parte dos ~15% de erros do modelo não são "erros grosseiros", mas confusões entre classes finas visualmente próximas — algo consistente com a granularidade do CIFAR-100 e que nenhuma das técnicas usadas neste trabalho tenta resolver diretamente (elas atacam overfitting geral, não a distinção fina entre classes semelhantes).



Exemplos classificados CORRETAMENTE





6. Conclusão

Cada decisão deste trabalho, a arquitetura larga em vez de profunda, o batch grande acoplado a uma lr mais alta com warmup e cosine decay, e o conjunto de técnicas de regularização e augmentation, foi motivada pela mesma restrição central: extrair o máximo de capacidade de generalização de uma rede de ~52M de parâmetros treinada com apenas 500 imagens por classe. O resultado de 85,08% de acurácia Top-1 e 97,03% Top-5, com métricas macro bem equilibradas, é consistente com a hipótese de que essas escolhas, combinadas, controlaram o overfitting de forma eficaz — evidenciado tanto pelo comportamento da curva de loss (queda abrupta ao desligar o MixUp/CutMix, coincidindo com a fase final do cosine decay) quanto pelo equilíbrio entre precision e recall no conjunto de teste.